

# Indoor Coverage Map Predictor

User Handbook & Blueprint

*Machine Learning for Indoor Cellular Coverage  
Prediction Using Ray Tracing*

**Zack Moyambo**

Professor Christopher Thron

|                      |                                                                                                         |
|----------------------|---------------------------------------------------------------------------------------------------------|
| <b>Purpose</b>       | Step-by-step guide to predicting indoor wireless coverage for any room using a pre-trained U-Net model. |
| <b>Audience</b>      | Anyone who wants to use the trained model — no machine learning knowledge required.                     |
| <b>Prerequisites</b> | A Google Account with Colab access. The trained model files saved to Google Drive.                      |
| <b>Time to run</b>   | Less than 5 minutes once the model is loaded.                                                           |

# Table of Contents

---

1. What This Tool Does
2. Before You Start — Prerequisites
3. Step-by-Step: Running the Prediction Notebook
4. Understanding the Room Definition
5. Wall Materials Reference
6. Transmitter Configuration
7. Understanding the Coverage Maps
8. Coverage Class Reference
9. Troubleshooting Common Errors
10. Tips for Better Predictions
11. Full Code Reference

## 1. What This Tool Does

This tool uses a trained deep learning model — specifically a U-Net encoder-decoder neural network — to predict how strong a wireless signal will be at every point inside a room or building. Instead of running a full physics-based ray-tracing simulation (which can take hours), the model predicts coverage in seconds by applying what it learned from 10,000 training simulations.

The model predicts two things simultaneously:

- **Signal strength map** — a heatmap of received power in dBm at every grid cell
- **Coverage class map** — each cell labelled as No Signal / Poor / Fair / Good / Excellent

Both maps are shown at three heights:

- 0.5 m — floor level (phone on a table)
- 1.5 m — typical phone height when held
- 2.5 m — elevated (wall-mounted devices)

**Tip:** You do not need to understand the neural network to use this tool. Just describe your room and run the notebook.

## 2. Before You Start — Prerequisites

You need the following before running the prediction notebook:

| Requirement                 | Details                                                                                         |
|-----------------------------|-------------------------------------------------------------------------------------------------|
| Google Account              | Free account is sufficient. Colab Pro gives faster runtimes.                                    |
| Google Colab                | Open <a href="https://colab.research.google.com">colab.research.google.com</a> in your browser. |
| Trained model file          | coverage_unet.keras — saved to your Google Drive by the training notebook.                      |
| Metadata file               | representation_metadata.json — saved alongside the model.                                       |
| USER_Predict_Coverage.ipynb | The prediction notebook. Upload to Colab or open from Drive.                                    |

### Where to find the model files:

After the training notebook finishes, it saves all files to Google Drive automatically. The default path is:

```
/content/drive/MyDrive/part4_outputs/coverage_unet.keras  
/content/drive/MyDrive/part4_outputs/coverage_clf_unet.keras  
/content/drive/MyDrive/dataset/dataset_processed/representation_metadata.json
```

**Note:** If your model files are in a different folder, update the `MODEL_DIR` and `METADATA_DIR` variables at the top of Cell 1 before running.

## 3. Step-by-Step: Running the Prediction Notebook

---

### Step 1 — Open the notebook

Go to [colab.research.google.com](https://colab.research.google.com). Click File → Upload notebook and select `USER_Predict_Coverage.ipynb`. Alternatively, if you saved it to Drive, open it from File → Open notebook → Google Drive.

### Step 2 — Select a runtime

Go to Runtime → Change runtime type. Select GPU (any type works — T4, L4, or A100). Click Save. A GPU is not strictly required but speeds up prediction.

### Step 3 — Run Cell 1 (Setup)

Click the play button on Cell 1 or press Shift+Enter. Colab will ask you to mount your Google Drive — click Allow. The cell will load your trained models and print a confirmation message. You should see a green checkmark next to each file path.

### Step 4 — Edit Cell 2 (Define Your Room)

This is the only cell you need to edit. Change the wall coordinates, materials, windows, doors, furniture, and transmitter to match the room you want to predict. See Sections 4–6 for detailed instructions on each parameter.

### Step 5 — Run Cells 3, 4, and 5

Run each cell in order. Cell 3 builds the input tensor and runs the model (a few seconds). Cell 4 produces the coverage map figures. Cell 5 prints a text summary of coverage class percentages.

### Step 6 — Save your results

The coverage maps are automatically saved as a PNG file to `/content/`. Download them from the Files panel on the left side of Colab (click the folder icon, then right-click the PNG and choose Download).

## 4. Understanding the Room Definition

Your room is defined using Python data classes in Cell 2. You do not need Python experience — just follow the pattern in the example and change the numbers to match your room. All coordinates are in metres. The origin (0, 0) is the bottom-left corner of your room. X increases to the right. Y increases upward.

### 4.1 Defining Walls

Each wall is defined by its start point, end point, material, and optional thickness. A rectangular room needs four walls. An L-shaped room needs six.

```
# Wall(start_point, end_point, material, thickness)
my_walls = [
    Wall(Point(0, 0), Point(8, 0), "concrete", thickness=0.20), # South wall
    Wall(Point(8, 0), Point(8, 6), "concrete", thickness=0.20), # East wall
    Wall(Point(8, 6), Point(0, 6), "concrete", thickness=0.20), # North wall
    Wall(Point(0, 6), Point(0, 0), "concrete", thickness=0.20), # West wall
    Wall(Point(4, 0), Point(4, 6), "drywall", thickness=0.10), # Interior
]
```

**Note:** Walls are numbered starting from 0. Wall 0 is the first item in the list, Wall 1 is the second, and so on. You need these numbers when placing windows and doors.

### 4.2 Defining Windows

A window is placed on a wall using its wall index, the offset from the start of that wall to where the window begins, and the window length. All measurements in metres.

```
# Window(wall_index, start_offset, length)
my_windows = [
    Window(wall_index=0, start_offset=1.5, length=2.0), # On South wall
    Window(wall_index=2, start_offset=2.0, length=1.5), # On North wall
]
```

### 4.3 Defining Doors

Doors are defined the same way as windows.

```
# Door(wall_index, start_offset, length)
my_doors = [
    Door(wall_index=4, start_offset=2.5, length=0.9), # Door in interior wall
]
```

### 4.4 Defining Furniture

Furniture is defined by its bottom-left corner coordinates, width (X direction), depth (Y direction), and material. Furniture blocks signal and creates shadowing.

```
# Furniture(bottom_left_point, width, depth, material)
my_furniture = [
    Furniture(Point(0.5, 0.5), width=2.0, depth=1.0, material="wood"),
    Furniture(Point(5.0, 1.0), width=1.5, depth=0.8, material="metal"),
]
```

## 5. Wall Materials Reference

The material you choose for each wall determines how much signal it blocks. Higher attenuation means more signal is lost as the radio wave passes through. Use the exact string values shown in the Key column below.

| Material Key   | Description               | Approx. Attenuation |
|----------------|---------------------------|---------------------|
| concrete       | Standard concrete wall    | ~20 dB              |
| thick_concrete | Extra thick concrete      | ~30 dB              |
| reinforced     | Reinforced concrete       | ~35 dB              |
| brick          | Brick wall                | ~15 dB              |
| drywall        | Interior drywall / gypsum | ~6 dB               |
| glass          | Standard window glass     | ~3 dB               |
| wood           | Wooden wall or door       | ~5 dB               |
| metal          | Metal door or panel       | ~40 dB              |

**Tip:** For a typical office or apartment with drywall interior partitions and concrete exterior walls, use "concrete" for outer walls and "drywall" for interior walls.

**Note:** Material names are case-sensitive. Use lowercase exactly as shown in the Key column.

## 6. Transmitter Configuration

---

The transmitter (TX) represents the base station, access point, or cell tower that is sending the signal into your room. You configure it with three parameters:

- **TX\_POSITION** — the [x, y, z] coordinates of the transmitter in metres. For outdoor-to-indoor scenarios, place the TX outside the room (negative x or y, or beyond the room boundary). For an indoor access point, place it inside the room.
- **TX\_FREQUENCY** — 700e6 for 700 MHz (LTE low-band) or 3.5e9 for 3.5 GHz (5G mid-band).
- **TX\_POWER\_DBM** — transmit power in dBm. 23 dBm is a small cell. 30 dBm is a stronger small cell.

### Examples:

```
# Outdoor macro tower 15m west of the room at 10m height
TX_POSITION = np.array([-15.0, 3.0, 10.0])
TX_FREQUENCY = 700e6
TX_POWER_DBM = 30.0

# Indoor WiFi access point mounted on the ceiling inside the room
TX_POSITION = np.array([4.0, 3.0, 2.8])
TX_FREQUENCY = 3.5e9
TX_POWER_DBM = 23.0
```

**Tip:** 700 MHz penetrates walls much better than 3.5 GHz. If you are simulating a building with thick concrete walls, 700 MHz will show much better indoor coverage.

## 7. Understanding the Coverage Maps

The output is a 3 x 2 grid of maps — three rows (one per height level) and two columns (signal strength on the left, coverage class on the right).

### 7.1 Signal Strength Maps (left column)

These are heatmaps showing received power in dBm at each grid cell. The colour scale runs from red (weak signal) through yellow to green (strong signal). The red star marks the transmitter position. Walls are overlaid as coloured lines. Windows appear as cyan lines.

- Darker green = stronger signal, better coverage
- Darker red = weaker signal, poor or no coverage
- Shadow regions behind thick walls will appear red even close to the transmitter

### 7.2 Coverage Class Maps (right column)

These maps classify each grid cell into one of five coverage categories based on the predicted dBm value. The colour key is shown in the legend on each map.

| Class     | dBm Range       | Meaning                    | Color  |
|-----------|-----------------|----------------------------|--------|
| No Signal | below -154 dBm  | No usable signal           | Red    |
| Poor      | -154 to -60 dBm | Very weak — calls may drop | Orange |
| Fair      | -60 to -50 dBm  | Usable but slow data       | Yellow |
| Good      | -50 to -43 dBm  | Reliable voice and data    | Green  |
| Excellent | above -43 dBm   | Full signal strength       | Blue   |

### 7.3 Grid Resolution

Each cell in the grid represents a 0.5 m x 0.5 m area. The model predicts one value per cell. The total number of cells depends on your room dimensions — an 8 m x 6 m room produces a 16 x 12 grid of predictions.



## 8. Coverage Class Reference

Coverage classes are determined by comparing the predicted dBm value against the thresholds below. These thresholds were calibrated to match the signal range produced by the Sionna ray-tracing simulations used to train the model.

| Class     | dBm Range       | Meaning                    | Color  |
|-----------|-----------------|----------------------------|--------|
| No Signal | below -154 dBm  | No usable signal           | Red    |
| Poor      | -154 to -60 dBm | Very weak — calls may drop | Orange |
| Fair      | -60 to -50 dBm  | Usable but slow data       | Yellow |
| Good      | -50 to -43 dBm  | Reliable voice and data    | Green  |
| Excellent | above -43 dBm   | Full signal strength       | Blue   |

**Note:** These thresholds reflect the specific training setup — outdoor small-cell transmitter at 23-30 dBm power. They may not match real-world carrier coverage thresholds (which typically use -85 dBm as the "Good" boundary for LTE). Treat the class labels as relative indicators within this simulation context.

## 9. Troubleshooting Common Errors

---

### **Error: FileNotFoundError: coverage\_unet.keras not found**

**Solution:** The model file is not in the path specified by MODEL\_DIR. Check that the training notebook completed successfully and saved the model to Drive. Then update MODEL\_DIR at the top of Cell 1 to point to the correct folder. Run the file check cell to confirm the path.

### **Error: FileNotFoundError: representation\_metadata.json not found**

**Solution:** The metadata file is missing. It should have been saved automatically by the training notebook to /content/drive/MyDrive/dataset/dataset\_processed/. Update METADATA\_DIR in Cell 1 if it is stored elsewhere.

### **Error: ValueError: Mountpoint must not already contain files**

**Solution:** Your Drive is already mounted. Replace drive.mount with drive.mount("/content/drive", force\_remount=True) to remount cleanly.

### **Error: NameError: name "GRID\_W" is not defined**

**Solution:** Cell 1 did not finish running successfully. Scroll up and check for error messages in Cell 1. Re-run Cell 1 before running any other cells.

### **Error: All predictions show the same class (e.g. all No Signal)**

**Solution:** The SIGNAL\_MEAN and SIGNAL\_STD used for denormalisation may not match the training values. Make sure representation\_metadata.json is from the same training run as coverage\_unet.keras. Do not mix model files from different training runs.

### **Error: Prediction runs but maps look blank or uniform**

**Solution:** Your room coordinates may be outside the grid bounds. Make sure wall coordinates start near (0,0) and do not exceed the grid dimensions (GRID\_W x GRID\_H cells at 0.5 m each, typically 60 x 30 cells = 30 m x 15 m maximum).

## 10. Tips for Better Predictions

---

### Start with the example room

Run the notebook with the default 8 m x 6 m room first to confirm everything works, then modify it to match your room.

### Keep coordinates starting at (0, 0)

Always place the bottom-left corner of your room at Point(0, 0). The model was trained on rooms anchored this way.

### Do not exceed the grid bounds

Keep your room dimensions within the model's grid size. Rooms larger than about 30 m x 15 m will be cropped. For larger rooms, increase GRID\_W and GRID\_H but note this may affect accuracy.

### Test multiple transmitter positions

Run the notebook multiple times with the TX in different positions to find the optimal placement for coverage. Each run takes only seconds.

### Compare 700 MHz vs 3.5 GHz

Run twice — once with TX\_FREQUENCY = 700e6 and once with TX\_FREQUENCY = 3.5e9 — to see how the two frequencies penetrate your room differently.

### Use metal furniture sparingly

Metal furniture blocks signal significantly (~40 dB). If your predictions show unexpected dead spots, check if metal furniture is blocking the path.

### Increase TX power for larger rooms

If you see mostly No Signal or Poor coverage, try increasing TX\_POWER\_DBM from 23 to 30 or 37. This simulates a more powerful transmitter.

## 11. Full Code Reference

This section shows the complete room definition template you can copy directly into Cell 2. Replace the example values with your own room dimensions.

```
# =====
# DEFINE MY ROOM – edit these values to match your room
# All measurements in metres. Origin (0,0) = bottom-left corner.
# =====

# -- WALLS -----
# Wall(Point(x_start, y_start), Point(x_end, y_end), material, thickness)
# Materials: concrete, thick_concrete, reinforced, brick, drywall, glass, wood, metal

my_walls = [
    Wall(Point(0, 0), Point(10, 0), "concrete", thickness=0.20), # South
    Wall(Point(10,0), Point(10, 8), "concrete", thickness=0.20), # East
    Wall(Point(10,8), Point(0, 8), "concrete", thickness=0.20), # North
    Wall(Point(0, 8), Point(0, 0), "concrete", thickness=0.20), # West
    # Add interior walls here:
    # Wall(Point(5, 0), Point(5, 8), "drywall", thickness=0.10),
]

# -- WINDOWS -----
# Window(wall_index, start_offset, length)
# wall_index = position of the wall in my_walls (0 = first wall)

my_windows = [
    Window(wall_index=0, start_offset=2.0, length=2.0),
    Window(wall_index=2, start_offset=3.0, length=2.0),
]

# -- DOORS -----
# Door(wall_index, start_offset, length)

my_doors = [
    Door(wall_index=1, start_offset=3.5, length=0.9),
]

# -- FURNITURE -----
# Furniture(Point(x, y), width, depth, material)
# Materials: wood, metal

my_furniture = [
    Furniture(Point(1.0, 1.0), width=2.0, depth=1.0, material="wood"),
    Furniture(Point(1.0, 6.0), width=1.5, depth=1.5, material="wood"),
]

# -- TRANSMITTER -----
TX_POSITION = np.array([-12.0, 4.0, 3.0]) # [x, y, z] in metres
TX_FREQUENCY = 3.5e9 # 700e6 = 700 MHz or 3.5e9 = 3.5 GHz
TX_POWER_DBM = 30.0 # transmit power in dBm

# -- ASSEMBLE -----
my_floorplan = Floorplan(
```

```
walls=my_walls, windows=my_windows,  
doors=my_doors, furniture=my_furniture,  
name="my_room"  
)
```

---

After editing Cell 2, run Cells 3, 4, and 5 in order. You can re-run as many times as you like with different room configurations — the model is already loaded and each prediction takes only a few seconds.

*For questions or issues, refer to Section 9 (Troubleshooting) or check the main research notebook for the full implementation details.*